

Planification d'itinéraires cyclistes

Rapport PAI2D

Alexandre Genin, Ron Sharony, Alex Thirault

2026

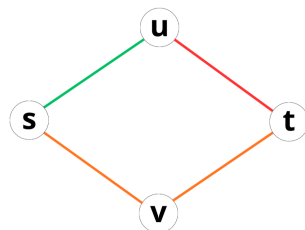
Table des matières

1	Introduction	1
2	État de l'art	2
3	Reformulation multi-objectif	3
4	Approche algorithmique	4
4.1	Algorithme de Dijkstra multi-objectif	4
4.1.1	Variante bidirectionnelle	4
4.2	Algorithme A* multi-objectif bidirectionnel	5
4.3	Heuristiques	5
4.3.1	Distance à vol d'oiseau	5
4.3.2	Distance avec des landmarks	6
4.3.3	Combinaison des heuristiques	7
4.4	Conditions d'arrêt	7
4.5	Réduction du nombre de solutions retournées	7
4.5.1	Élagage en fonction de la distance totale d'un chemin	7
4.5.2	Ajout de contraintes ordinales	7
4.6	Pseudo-code	9
5	Application développée	10
6	Résultats expérimentaux	11
6.1	Condition d'arrêt	11
6.2	Heuristiques	11
6.3	Jeux de poids	12

1 Introduction

Les applications de planification d'itinéraires sont des services particulièrement courants de nos jours. Avec l'essor des déplacements à vélo, diverses applications spécialement dédiées aux cycles se sont développées, parmi lesquelles Geovelo, ou encore Naviki.

Dans ce projet, nous proposons un nouvel outil de planification d'un itinéraire cycliste qui tient compte du niveau de sécurité des voies empruntées (lié par exemple à la présence d'une piste cyclable, à l'absence de carrefour mal protégé, etc.) : c'est une caractéristique de nature ordinale qui doit être manipulée avec vigilance. En effet, assigner des valeurs numériques quelconques aux catégories représentant les niveaux de sécurité introduit vite des biais. Considérons l'instance suivante :



En admettant que les arcs verts sont très sûrs, les arcs oranges moyennement sûrs et les arcs rouges dangereux, on peut proposer d'assigner 1 au vert, 4 à l'orange et 8 au rouge. On voudrait donc minimiser

la dangerosité de notre chemin. Avec cette assignation, le chemin préféré de s à t serait celui du bas, qui emprunte deux arcs oranges et a ainsi une dangerosité (en supposant qu'elle est additive) de 8 au lieu de celui du haut de dangerosité 9. Cette assignation est acceptable car on y préfère les arcs verts aux oranges aux rouges. Mais assigner 1 au vert, 5 à l'orange et 8 au rouge est également acceptable et pourtant, cela nous amènerait à préférer le chemin du haut de dangerosité 9 à celui du bas de dangerosité 10. Un léger changement des valeurs numériques assignées aux catégories amène à des choix très différents et notre reformulation multi-objectif permettra de résoudre ce problème.

Bien que ce critère de choix sur les niveaux de sécurité soit proposé dans certains outils déjà existants, la méthode construite repose sur une approche originale, avec une modélisation multi-objectif où chaque objectif représente le nombre de kilomètres du trajet respectant un certain niveau de sécurité. L'application développée retourne un ensemble d'itinéraires possibles, réalisant différents compromis temps de trajet/sécurité.

2 État de l'art

Présentons ce qui est déjà implémenté chez Geovelo.

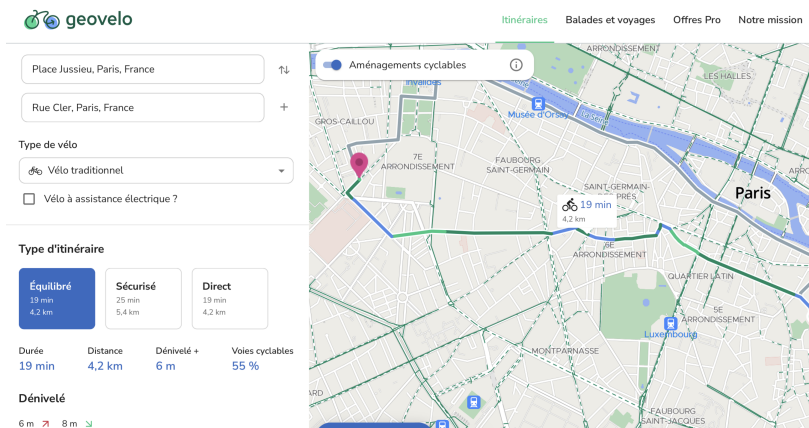


FIGURE 1 – Itinéraire proposé sur Geovelo entre la Place Jussieu et la Rue Cler

L'utilisateur choisit une adresse de départ, une adresse de destination et Geovelo propose alors trois itinéraires distincts : un équilibré, un sécurisé et un direct. Quant aux critères sur les itinéraires, Geovelo mentionne :

- les pistes cyclables et voies dédiées (voies comportant un aménagement cyclable séparé de la circulation automobile),
- les double-sens cyclables,
- les voies cyclables partagées (voies comportant un aménagement cyclable partagé avec la circulation automobile (bandes, voies de bus)),
- les voies au trafic apaisé (voies avec une circulation automobile inférieure à 30km/h ou voies partagée avec les piétons),
- les zones temporairement bloquées aux cyclistes (les cyclistes ne pourront pas circuler dans ces zones et seront invités à emprunter des itinéraires de contournement),
- et enfin, les principaux axes cyclables (axes permettent de relier les principaux pôles d'activité du territoire, avec des aménagements adaptés aux cyclistes).

Bien sûr, la durée, la distance, le dénivelé de chacun des trajets sont mentionnés. Les voies empruntées par l'itinéraire le sont aussi. Notons que l'application offre le choix de préciser le type de vélo utilisé.

Naviki offre moins de liberté à l'utilisateur en proposant un seul trajet similaire au trajet équilibré de Geovelo.

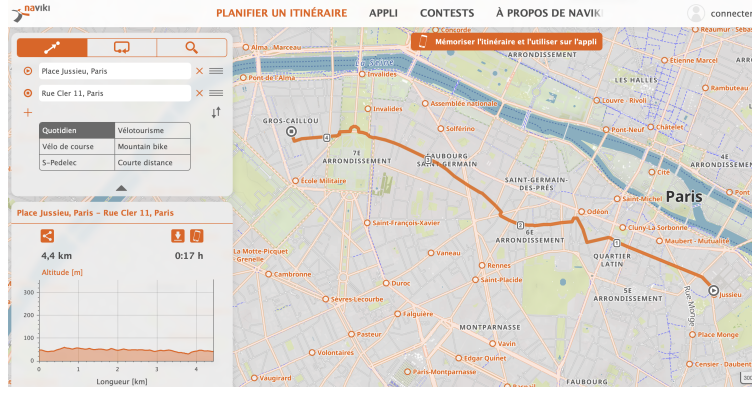


FIGURE 2 – Itinéraire proposé sur Naviki entre la Place Jussieu et la Rue Cler

3 Reformulation multi-objectif

Il existe une reformulation multi-objectif de notre problème d'optimisation ordinale, et nous explicitons donc les outils usuels dont nous allons avoir besoin.

D'abord, classiquement, la dominance stricte de Pareto :

Définition 1. Le vecteur $(a_1, \dots, a_n)$ domine le vecteur $(b_1, \dots, b_n)$ si et seulement si :

$$(\forall i \in \{1, \dots, n\} : a_i \leq b_i) \wedge (\exists i \in \{1, \dots, n\} : a_i < b_i)$$

Par ailleurs, on note $G = (V, A)$ un graphe orienté G où V est l'ensemble de ses sommets et A l'ensemble de ses arcs. Chaque arc est muni d'un vecteur décrivant sa distance ainsi que sa catégorie i.e. son niveau de sécurité. Un chemin est représenté par son vecteur de coût pour tous les objectifs (la distance sous une certaine catégorie) et par la liste des sommets dont il est constitué. Un label $Label$ est alors défini par $Label = [v, \bar{c}, Prev_L]$ où v est le sommet auquel le label est associé, $\bar{c}$ le vecteur de coût du chemin arrivant en v passant avant par le sommet de label $Prev_L$.

Définition 2. (Vecteur de coût cumulé) Soit un vecteur de coût $\bar{c}$ associé à un chemin P , noté $\bar{c} = (n_1(P), n_2(P), \dots, n_K(P))$, où $n_k(P)$ désigne le nombre de mètres de catégorie k empruntés par le chemin P .

On définit le vecteur de coût cumulé associé à un niveau k comme le vecteur :

$$\bar{C} = (C_1(P), C_2(P), \dots, C_K(P)),$$

où

$$C_k(P) = \sum_{i \leq k} n_i(P).$$

Ainsi, $C_k(P)$ représente le nombre total de mètres du chemin P de niveau de sécurité k ou inférieur.

Lorsque l'on manipule des critères ordinaux, deux dominances équivalentes apparaissent souvent : la dominance monotone et la dominance ordinale. Par ailleurs, en notant S et S' deux ensembles dont les éléments sont assignés à une catégorie, on note pour un élément i de S γ^i sa catégorie, et, pour une catégorie c , $S_c = \{i \in S : \gamma^i \leq c\}$ tous les éléments de S dont la catégorie est préférée à c . Nous manipulerons ici la dominance monotone définie comme suit afin de justifier la pertinence du vecteur de coût cumulé et ainsi de résoudre le problème mentionné en introduction.

Définition 3. [3] La dominance monotone (deux à deux) est définie, pour S, S' , par $S < S'$ s'il existe une injection $\pi : S' \rightarrow S$ telle que $\forall i \in S', \gamma^{\pi(i)} \leq \gamma^i$.

Proposition 3.1. [3] $\forall S, S' \subseteq \mathcal{N}$,

$$S \leq S' \iff (|S_1|, \dots, |S_C|) \preceq (|S'_1|, \dots, |S'_C|).$$

Démonstration. [3] On montre d'abord $S \leq S' \Rightarrow (|S_1|, \dots, |S_C|) \preceq (|S'_1|, \dots, |S'_C|)$. Supposons qu'il existe $\pi : S' \rightarrow S$. Ainsi, $|S_c| \geq |\pi(S'_c)| = |S'_c|$ pour tout c , puisque $\gamma^{\pi(i)} \leq \gamma^i$ pour tout $i \in \{1, \dots, n\}$. Donc $(|S_1|, \dots, |S_C|) \preceq (|S'_1|, \dots, |S'_C|)$ par définition de la dominance de Pareto.

On montre maintenant la réciproque $(|S_1|, \dots, |S_C|) \preceq (|S'_1|, \dots, |S'_C|) \Rightarrow S \leq S'$. Supposons $|S_c| \geq |S'_c|$ pour tout c . Puisque $|S_1| \geq |S'_1|$, il existe une injection $\pi_1 : S'_1 \rightarrow S_1$. On a bien sûr $\forall i \in S'_1, \gamma^{\pi_1(i)} \leq \gamma^i$. Pour tout $c > 1$, on peut ainsi définir de manière récursive :

- une injection $\pi'_c : S'_c \setminus S'_{c-1} \rightarrow S_c \setminus \pi_{c-1}(S'_{c-1})$
 - une injection $\pi_c : S'_c \rightarrow S_c$ par $\pi_c(i) = \pi_{c-1}(i)$ si $i \in S'_{c-1}$ et $\pi_c(i) = \pi'_c(i)$ sinon.
- L'injection π'_c existe pour tout $c > 1$ car $|S_c \setminus \pi_{c-1}(S'_{c-1})| \geq |S'_c \setminus S'_{c-1}|$. On a en effet

$$|S_c \setminus \pi_{c-1}(S'_{c-1})| = |S_c| - |\pi_{c-1}(S'_{c-1})| = |S_c| - |S'_{c-1}| \geq |S'_c| - |S'_{c-1}| = |S'_c \setminus S'_{c-1}|$$

puisque π_{c-1} est injective.

Par construction, pour tout c , $\forall i \in S'_c$, $\gamma^{\pi_c(i)} \leq \gamma^i$. Pour $c = C$, c'est en fait la définition de préférer une solution à une autre, d'où $S \leq S'$. $\square$

En remarquant la similarité entre le vecteur $\bar{C}$ et $(|S_1|, \dots, |S_C|)$, on comprend donc pourquoi la reformulation multi-objectif du problème par le vecteur de coût cumulé est pertinente. Ainsi, un chemin P_1 domine un autre chemin P_2 , c'est-à-dire on préfère le chemin ordinal P_1 au chemin ordinal P_2 si et seulement si le vecteur de coût cumulé de P_1 domine celui de P_2 .

De la même manière, un label $Label_1$ domine un autre label $Label_2$ si et seulement si le vecteur de coût cumulé définissant $Label_1$ domine celui de $Label_2$.

Définition 4. (*Optimalité au sens de Pareto*)

Un chemin est dit Pareto optimal si et seulement s'il n'existe aucun autre chemin réalisable qui soit meilleur pour l'un des objectifs et non moins bon pour tous les autres objectifs.

Un ensemble de chemins S est dit Pareto optimal si et seulement si aucun des éléments de S n'est dominé par un autre chemin réalisable.

Cela se traduit sur l'exemple donné en introduction par les vecteurs cumulatifs $(2, 1, 1)$ et $(2, 2, 0)$ respectivement associés au chemin du haut et au chemin du bas. Cela nous donne donc l'ensemble des deux chemins qui est Pareto optimal car ni l'un ni l'autre se dominent.

À plusieurs reprises nous aurons besoin d'un ordre sur les labels afin de faire un choix sur ces derniers. Dans le but de déterminer le label le plus petit ou le plus grand, on se sert de l'ordre lexicographique ainsi défini :

Définition 5. (*Ordre lexicographique*)

Le vecteur $[a_1, \dots, a_n]$ est lexicographiquement plus petit que (noté $<_l$) le vecteur $[b_1, \dots, b_n]$ si et seulement si :

$$\exists k \in \{1, \dots, n\} : (\forall i < k : a_i = b_i) \wedge (a_k < b_k)$$

Remarque 1. *Cette relation est une relation d'ordre totale, car elle est réflexive, antisymétrique et transitive, et deux éléments peuvent toujours être comparés entre eux.*

L'ordre lexicographique n'est pas forcément le meilleur ordonnancement pour les algorithmes de plus court chemin multi-objectif. Dans l'étude de Paixão et Santos (2007) [2], il est démontré qu'un ordonnancement par agrégat de somme pondérée permet une réduction non négligeable du temps de calcul. Par souci de facilité, on se sert quand même de l'ordre lexicographique.

4 Approche algorithmique

4.1 Algorithme de Dijkstra multi-objectif

Martins présente en 1984 un algorithme, basé sur celui de Dijkstra, de marquage de labels afin de traiter le problème du plus court chemin multi-objectif où est associé à chaque sommet un ensemble de labels [5]. Chaque ensemble n'est constitué que de labels non dominés. Ainsi, on n'ajoute un nouveau label à un sommet que si celui-ci n'est pas dominé par un des labels déjà existants. De même, les labels dominés par celui que l'on ajoute sont supprimés : cela permet de ne pas avoir une explosion du nombre de chemins que l'on étudie. Mais cet algorithme s'avère être trop lent pour de grands réseaux comme les réseaux de transports, et ainsi, trop lent pour notre problème de planification d'itinéraires cyclistes.

4.1.1 Variante bidirectionnelle

Pour pallier cet exact problème, Demeyer, Goedgebeur, Audenaert, Pickavet et Demeester (2013) présentent dans *Speeding up Martins' algorithm for multiple objective shortest path problems* [4] un algorithme bidirectionnel, où la procédure usuelle de recherche est séparée en une procédure avant (partant du sommet source) et une procédure arrière (partant du sommet destination dans le graphe inversé), pour le problème du plus court chemin multi-objectif. On note alors que chaque sommet a désormais deux ensembles de labels : un ensemble avant et un ensemble arrière.

L'efficacité des algorithmes bidirectionnels pour les problèmes de plus courts chemins dépend de la manière dont on alterne entre les recherches avant et arrière ainsi que, principalement, de la condition d'arrêt. Pour un ensemble T de labels, on pose $\min(T) = [\min_1(T), \dots, \min_n(T)]$ le vecteur correspondant au minimum des coûts sur tous les labels de T pour chacun des n objectifs. On comparera les chemins de l'ensemble des chemins solutions au vecteur $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$.

Proposition 4.1. *Les recherches avant et arrière peuvent être arrêtées dès lors que $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$ est dominé par un chemin de l'ensemble des chemins solutions.*

4.2 Algorithme A* multi-objectif bidirectionnel

L'algorithme A* (Hart, Nilsson et Raphael, 1968) [7] constitue une généralisation de l'algorithme de Dijkstra intégrant une fonction heuristique, ce qui permet d'accélérer la recherche de plus courts chemins. Pour cette raison, nous avons choisi d'implémenter une variante multi-objectif et bidirectionnelle de l'algorithme A*.

Nous rappelons ci-dessous plusieurs notions importantes relatives à cet algorithme.

Définition 6.

- Une heuristique est une fonction $h : V \rightarrow \mathbb{R}$ estimant le coût restant d'un sommet donné jusqu'à un sommet objectif. Dans le cas de la recherche avant, l'objectif est le sommet destination. A l'inverse, la recherche arrière a pour objectif le sommet source.
- Soit $d^*(n, m)$ le coût d'un chemin optimal entre deux nœuds n et m (et $+\infty$ s'il n'existe pas de chemin).
- On définit $h^*(v)$ comme le coût du meilleur chemin allant de n à la source s (recherche arrière : $d^*(n, s)$ dans le graphe inversé) ou de n à la destination t (recherche avant : $d^*(n, t)$).
- On définit $g^*(v)$ comme le coût du meilleur chemin allant de la source s à v (recherche avant : $d^*(s, v)$) ou de v à la destination t (recherche arrière : $d^*(t, v)$ dans le graphe inversé) comme le coût optimal de v vers un sommet but.
- La fonction d'évaluation optimale est donnée par :

$$f^*(v) = g^*(v) + h^*(v).$$

- En pratique, cette quantité est estimée par :

$$f(v) = g(v) + h(v),$$

où $g(v)$ est le coût du meilleur chemin connu de s à v , et $h(v)$ une heuristique estimant de manière optimiste le coût restant jusqu'à un objectif.

4.3 Heuristiques

Pour garantir que l'algorithme A* renvoie des chemins Pareto-optimaux, l'heuristique choisie doit être admissible (ou minorante).

Définition 7. (Heuristique admissible)

Une heuristique est dite admissible si h ne surestime jamais la distance réelle h^* entre deux sommets du graphe :

$$\forall v \in V, \quad h(v) \leq h^*(v)$$

Hormis l'admissibilité de l'heuristique, il est important de s'intéresser à sa qualité puisqu'elle influence l'efficacité computationnelle de l'algorithme A*. En effet, une heuristique pertinente élague efficacement des labels en fonction de la distance totale maximale autorisée (voir section 4.5.1). Elle guide également la recherche, réduisant ainsi l'amplitude de la zone explorée. Néanmoins, une heuristique plus informative qu'une autre n'implique pas nécessairement son éléction. Un gain d'information peut cacher des calculs plus coûteux. Par conséquent, il faut sélectionner une heuristique non seulement informative, mais aussi calculable en une durée raisonnable et nécessitant peu de mémoire.

4.3.1 Distance à vol d'oiseau

La première heuristique choisie est la distance à vol d'oiseau entre un sommet courant et la destination. Elle correspond à une relaxation classique du problème du plus court chemin.

Calculée en temps constant et facilement implémentable, elle se révèle particulièrement efficace pour des trajets courts ou des graphes denses où la distance réelle est fortement corrélée à la distance estimée.

Nous utilisons ici la formule de Haversine [9] pour obtenir la distance entre deux points à la surface de la Terre à partir de leurs coordonnées géographiques, tout en tenant compte de la courbure de la planète.

Définition 8. (Heuristique "distance à vol d'oiseau") Soient (φ_1, λ_1) la latitude et la longitude d'un sommet v du graphe, et (φ_2, λ_2) celles du sommet cible.

L'heuristique h_1 est définie par :

$$h_1(v) = 2R \cdot \arctan 2 \left(\sqrt{a}, \sqrt{1-a} \right)$$

$$\text{où } a = \sin^2 \left(\frac{\varphi_2 - \varphi_1}{2} \right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2 \left(\frac{\lambda_2 - \lambda_1}{2} \right).$$

La formule de Haversine calcule une distance sur une sphère. Nous supposons donc ici que la Terre peut être approximée par une sphère, bien qu'elle ait une forme ellipsoïdale. Par ailleurs, la surface considérée est supposée lisse. Puisque la formule ne prend donc pas en compte les reliefs, les cartes utilisées pour illustrer les performances de notre algorithme correspondent à des villes à faible relief, e.g. Paris.

4.3.2 Distance avec des landmarks

Pour les grandes distances ou les graphes où la distance à vol d'oiseau ne suffit pas pour appréhender l'organisation du réseau routier, nous utilisons une seconde heuristique basée sur le concept de *landmarks* (ou points de repère). Nous nous appuyons ici sur les travaux de Goldberg et Harrelson (2005) qui introduisent l'algorithme *ALT* (*A**, *Landmarks*, *Triangle inequality*) dans leur article *Computing the shortest path : A search meets graph theory* [6]. Cette méthode repose sur la sélection d'un ensemble L de sommets appelés *landmarks*, où les distances depuis et vers chaque *landmark* dans le graphe sont pré-calculées. Ces informations permettent ensuite de fournir une estimation de la distance réelle en exploitant les inégalités triangulaires.

Prétraitement Contrairement à h_1 , l'heuristique h_2 est construite en partie grâce à des calculs préliminaires à toute requête. Le coût de ce pré-traitement peut être justifié en pratique par la réutilisation d'une même carte pour des requêtes de plus courts chemins.

Pour chaque sommet v du graphe G , les plus courtes distances de v à chaque *landmark* (et vice-versa) sont calculées à l'aide de l'algorithme de Dijkstra, puis stockées en mémoire. La complexité spatiale du procédé est donc linéaire en la taille du graphe et le nombre de *landmarks* choisis : $\mathcal{O}(|V| \cdot |L|)$. Quant à sa complexité temporelle, nous rappelons que l'algorithme de Dijkstra implémenté avec un tas binaire s'exécute en $\mathcal{O}((|V| + |E|) \log(|V|))$. Les distances étant calculées depuis chaque *landmark* dans le graphe original et dans le graphe inversé, la complexité temporelle finale est en $\mathcal{O}(|L| \cdot (|V| + |E|) \log |V|)$. Ainsi, ces coûts restent acceptables compte tenu de l'usage visé.

Estimation heuristique En s'appuyant sur l'inégalité triangulaire inversée et à l'aide des distances précédemment obtenues, il est alors possible de construire une heuristique admissible pour l'algorithme A^* .

Définition 9. (Heuristique "landmarks") Soient L un ensemble de *landmarks*, v un sommet du graphe, s le sommet source et t le sommet destination.

Pour la recherche avant, l'inégalité triangulaire nous donne : $\forall l \in L$,

$$d^*(v, t) \geq d^*(l, t) - d^*(l, v)$$

$$d^*(v, t) \geq d^*(v, l) - d^*(t, l)$$

$$\text{d'où } h_2(v) = \max_{l \in L} \left(d^*(l, t) - d^*(l, v), d^*(v, l) - d^*(t, l) \right)$$

Pour la recherche arrière (pour les distances exprimées dans le graphe original) : $\forall l \in L$,

$$d^*(s, v) \geq d^*(s, l) - d^*(v, l)$$

$$d^*(s, v) \geq d^*(l, v) - d^*(l, s)$$

$$\text{d'où } h_2(v) = \max_{l \in L} \left(d^*(s, l) - d^*(v, l), d^*(l, v) - d^*(l, s) \right)$$

Pour une requête particulière, l'estimation de la distance restante s'effectue alors en temps linéaire en le nombre de *landmarks*. Le nombre de *landmarks* étant généralement faible, ce coût reste raisonnable.

Choix des landmarks Les *landmarks* sont choisis de manière à maximiser leur répartition dans le graphe, ce qui améliore la qualité des bornes inférieures. Ici, nous nous inspirons de l'idée de la *planar landmark selection* (sélection planaire des points de repère) présentée dans le même article. Bien qu'il existe des techniques permettant de déterminer automatiquement des points de repère, nous avons choisi nos *landmarks* manuellement, tout en respectant le principe de la sélection planaire : quelques points centraux et de nombreux points de la périphérie uniformément espacés.

4.3.3 Combinaison des heuristiques

Dans notre implémentation, les deux heuristiques sont combinées. Ainsi, l'algorithme sélectionne l'heuristique la plus pertinente en prenant le maximum des deux estimations :

$$h(v) = \max(h_1(v), h_2(v))$$

Cette stratégie garantit la conservation de l'admissibilité tout en améliorant la précision des estimations, ce qui réduit le nombre de labels explorés.

4.4 Conditions d'arrêt

Tout comme l'algorithme de Dijkstra multi-objectif bidirectionnel, les performances de cet algorithme dépendent en partie du choix de sa condition d'arrêt. Nous avons donc considéré et testé quatre conditions différentes lors de notre travail (voir section 6) :

- **1. stop1** correspond à la condition précédemment décrite pour l'algorithme de Dijkstra multi-objectif bidirectionnel (voir section 4.1.1). Elle arrête la recherche dès que le vecteur idéal (somme des minima forward et backward) est dominé par au moins une solution Pareto-optimale déjà trouvée.
- **2. La condition stop2** est plus stricte : elle n'arrête que si, *pour tous* les labels temporaires forward, la combinaison de ce label avec la borne inférieure backward (vecteur idéal T_{minB}) est dominée par une solution connue. Autrement dit, on vérifie qu'aucun chemin partiel forward, même complété par le meilleur arrière possible, ne peut améliorer le front de Pareto.
- **3. stop3** affine stop2 en remplaçant, pour le premier objectif (généralement la distance), la composante backward par une distance à vol d'oiseau entre le sommet courant du label forward et la destination. Cette heuristique géographique fournit une borne inférieure plus réaliste que le minimum des labels backward, rendant la condition d'arrêt potentiellement plus précoce.
- **4. Enfin, stop4** considère *toutes les paires* de labels temporaires forward/backward. Pour chaque label forward, on combine son vecteur avec la distance à vol d'oiseau (au lieu du vecteur backward sur le premier objectif) et avec le vecteur backward complet sur les autres objectifs. Si toutes ces combinaisons sont dominées par les solutions connues, la recherche s'arrête. Cette condition est la plus conservatrice (arrêt le plus rapide) mais aussi la plus coûteuse en calcul.

4.5 Réduction du nombre de solutions retournées

Le nombre de solutions Pareto-optimales renvoyées par l'algorithme croît significativement à mesure que la distance augmente entre la source et la destination. Nous souhaitons donc restreindre la liste de chemins proposés en s'appuyant sur des critères autres que la Pareto-optimalité.

4.5.1 Élagage en fonction de la distance totale d'un chemin

Dans un premier temps, nous proposons l'introduction d'une simple borne supérieure qui permet de rejeter des solutions en fonction de leur longueur totale. En effet, dans un cadre applicatif, un utilisateur est prêt à parcourir de la distance supplémentaire pour gagner en sécurité mais seulement si la longueur du détour n'est pas déraisonnablement élevée. Pour ce faire, l'algorithme A* est appliqué une première fois dans le cas mono-objectif afin de trouver le plus court chemin entre les sommets source et destination. Une fois cette distance minimale obtenue, il nous est alors possible d'écarter des solutions si leur distance totale (réelle ou estimée) dépasse un certain seuil de cette borne inférieure.

Lors de la phase de recherche, un sommet v est exploré seulement si :

$$g(v) + h(v) \leq d^*(s, t) \cdot \left(1 + \frac{seuil}{100}\right)$$

Lors de la combinaison des solutions (recherches avant et arrière), seules les solutions vérifiant l'inégalité suivante sont conservées :

$$d(s, v) + d(v, t) \leq d^*(s, t) \cdot \left(1 + \frac{seuil}{100}\right)$$

4.5.2 Ajout de contraintes ordinales

La technique d'élagage des chemins présentée ci-dessus prend seulement en compte la distance totale d'une solution et non la sécurité de ses tronçons de route. Pourtant, en pratique, il est courant que l'utilisateur exprime une préférence pour un chemin Pareto-optimal plutôt qu'un autre, en fonction de ses jugements vis-à-vis des différentes catégories de sécurité proposées. Illustrons ce phénomène avec l'exemple suivant :

Exemple 1. Soit le graphe $G^I = (V^I, E^I)$ avec $V^I = \{v_1, v_2, v_3\}$, $E^I = \{(v_1, v_2), (v_2, v_3), (v_1, v_3)\}$ et la fonction de coût d'arêtes c telle que : $c((v_1, v_2)) = [10, 0, 0]$, $c((v_2, v_3)) = [10, 0, 0]$, $c((v_1, v_3)) = [0, 0, 18]$, où un vecteur représente la distance totale en unités des tronçons par niveau de sécurité entre deux points.

Il existe deux chemins Pareto-optimaux pour se déplacer de v_1 à v_3 : 1. $v_1 \rightarrow v_2 \rightarrow v_3$, empruntant 20 unités de routes de catégorie A (sécurité élevée), et 2. $v_1 \rightarrow v_3$, parcourant 18 unités sur un tronçon de classe C (sécurité faible). Ici, bien que les vecteurs de coût associés à ces deux chemins soient tous deux Pareto-optimaux, le chemin 1, qui implique un léger détour, peut être largement préféré au chemin 2, qui est plus court mais bien plus dangereux.

A contrario, si par exemple le premier chemin est constitué de 20 unités de tronçons de catégorie B, ce niveau de sécurité pourrait ne pas justifier le détour, et la solution 2 pourrait alors être choisie.

L'exemple décrit permet de comprendre qu'il s'agit ici de quantifier dans quelle mesure une catégorie est perçue comme « meilleure » ou « pire » qu'une autre par l'utilisateur.

Dominance ordinale Pour formaliser ces concepts, nous reprenons les définitions proposées par Klamroth et al. (2026) dans leur article *Adapting Polyhedral Dominance Cones to Ordinal Preference Structures* [8].

Soit K le nombre de catégories de sécurité. On suppose qu'une catégorie η_i est strictement préférée à la catégorie η_{i+1} pour toutes les catégories. Afin de définir la dominance ordinale, nous introduisons les deux vecteurs de préférence suivants.

Définition 10. (Vecteurs ω et γ) Soient $\omega, \gamma \in \mathbb{R}^{K-1}$ deux vecteurs tels que $\omega_i \geq 1$ et $\gamma_i \geq 0$ pour tout $i \in \{1, \dots, K-1\}$.

Ces vecteurs modélisent l'intensité des préférences entre catégories consécutives η_i et η_{i+1} .

Le vecteur ω représente le nombre d'éléments de la catégorie η_i considérés comme préférés à un élément de la catégorie η_{i+1} .

Le vecteur γ représente la préférence inverse en indiquant combien d'éléments de η_{i+1} sont jugés moins bons qu'un élément de η_i .

Remarque 2. Afin de distinguer les catégories η_i et η_{i+1} , il est évident qu'il faut que $\omega_i \gamma_i < 1$.

Définition 11. (Représentation numérique (strictement) standardisée relative à (ω, γ)) Un vecteur $\nu = (\nu_1, \nu_2, \dots, \nu_K) \in (\mathbb{R}_{>0})^K$ est une (ω, γ) -représentation numérique strictement standardisée si :

$$\omega_i \nu_i < \nu_{i+1}, \quad \forall i \in \{1, \dots, K-1\}$$

$$\nu_i > \gamma_i \nu_{i+1}, \quad \forall i \in \{1, \dots, K-1\}$$

De façon similaire, un vecteur $\nu = (\nu_1, \nu_2, \dots, \nu_K) \in (\mathbb{R}_{>0})^K$ est une (ω, γ) -représentation numérique standardisée si :

$$\omega_i \nu_i \leq \nu_{i+1}, \quad \forall i \in \{1, \dots, K-1\}$$

$$\nu_i \geq \gamma_i \nu_{i+1}, \quad \forall i \in \{1, \dots, K-1\}$$

Nous notons $V_{(\omega, \gamma)}^>$ l'ensemble des (ω, γ) -représentations numériques strictement standardisées et $V_{(\omega, \gamma)}^\geq$ l'ensemble des (ω, γ) -représentations numériques standardisées. Dans notre contexte, ce dernier ensemble définit un polyèdre de pondérations admissibles.

Remarque 3. La structure ordinale de dominance standard est obtenue pour $\omega = \mathbf{1} := (1, \dots, 1)^\top \in \mathbb{R}^{K-1}$ et $\gamma = \mathbf{0} := (0, \dots, 0)^\top \in \mathbb{R}^{K-1}$, c'est-à-dire $V_o = V(\mathbf{1}, \mathbf{0})$.

Définition 12. (Dominance ordinale relative à (ω, γ))

Soient $x', \hat{x} \in X$ (ensemble des chemins possibles) deux solutions et $c(x')$, $c(\hat{x})$ leurs vecteurs de coût respectivement associés.

1. On dit que x' (ω, γ) -domine ordinalement faiblement $\hat{x}$, noté $x' \preceq_{(\omega, \gamma)} \hat{x}$ (ou de manière équivalente $c(x') \preceq_{(\omega, \gamma)} c(\hat{x})$), si et seulement si, pour tout $\nu \in V_{(\omega, \gamma)}^\geq$, on a :

$$\nu^\top c(x') \leq \nu^\top c(\hat{x}).$$

2. On dit que x' (ω, γ) -domine ordinalement $\hat{x}$, noté $x' \preceq_{(\omega, \gamma)} \hat{x}$, et que $c(x') \preceq_{(\omega, \gamma)} c(\hat{x})$, si et seulement si :

- x' (ω, γ) -domine faiblement $\hat{x}$, et
- il existe $\nu^* \in V(\omega, \gamma)$ tel que

$$\nu^{*\top} c(x') < \nu^{*\top} c(\hat{x}).$$

3. Un point $x^* \in X$ est dit (ω, γ) -ordinalement efficace s'il n'existe pas $x \in X$ tel que $x \preceq_{(\omega, \gamma)} x^*$.

Remarque 4. Dans le cas particulier où $\omega = \mathbf{1}$ et $\gamma = \mathbf{0}$, la dominance ordinale associée à (ω, γ) s'interprète comme une dominance de Pareto classique appliquée au vecteur de coût cumulé.

Points extrêmes du polyèdre des jeux de poids Sachant que $V_{(\omega, \gamma)}^{\geq}$ est le polyèdre des pondérations (représentations numériques standardisées) admissibles, l'évaluation de la dominance ordinale entre deux vecteurs de coût peut se ramener à uniquement vérifier les inégalités pour les vecteurs extrêmes (points) de ce polyèdre. Formellement, cela s'exprime de la façon suivante.

Proposition 4.2. *Soit $V_{(\omega, \gamma)}^{\geq}$ le polyèdre des pondérations admissibles et $\text{Ext}(V_{(\omega, \gamma)}^{\geq})$ l'ensemble de ses points extrêmes. Alors, pour $x', \hat{x} \in X$:*

$$\nu^{\top} c(x') \leq \nu^{\top} c(\hat{x}), \quad \forall \nu \in V_{(\omega, \gamma)}^{\geq}$$

si et seulement si

$$\nu^{\top} c(x') \leq \nu^{\top} c(\hat{x}), \quad \forall \nu \in \text{Ext}(V_{(\omega, \gamma)}^{\geq}).$$

Afin de déterminer ces points extrêmes, nous utilisons une formule présentée par Ahn et Park (2014) dans leur article *Establishing dominance between strategies with interval judgments of state probabilities* [1]. Il suffit de fixer pour chaque paire de catégories voisines, soit la borne inférieure soit la borne supérieure des rapports de poids. Chaque point correspond donc à un choix de bornes pour tous les rapports entre poids adjacents.

On obtient ainsi le résultat suivant, reposant sur la Proposition 3 de [1] :

Proposition 4.3. *Soit un ensemble de contraintes d'intervalles définissant $V_{(\omega, \gamma)}^{\geq}$. Alors :*

1. *Le nombre total de points extrêmes du polyèdre $V_{(\omega, \gamma)}^{\geq}$ est de l'ordre de 2^{K-1} .*
2. *Chaque point extrême est obtenu en résolvant le système linéaire suivant :*

$$\frac{\nu_j}{\nu_{j+1}} = \ell_j \text{ ou } u_j, \quad 1 \leq j \leq K-1,$$

$$\sum_{j=1}^K \nu_j = 1,$$

où ℓ_j et u_j sont les bornes inférieure et supérieure des ratios entre catégories adjacentes. Dans notre cadre, les bornes inférieures correspondent au vecteur $1/\omega$ et les bornes supérieures au vecteur $1/\gamma$.

4.6 Pseudo-code

En se basant sur l'approche algorithme présentée ci-dessus, on a le pseudo-code suivant :

```

1: distanceMonoObjectif  $\leftarrow$  Dijkstra(source, dest)
2: distanceMax  $\leftarrow$   $(1 + \frac{\text{seuil}}{100}) \cdot \text{distanceMonoObjectif}$ 

3: T[avant]  $\leftarrow$   $\emptyset$ 
4: T[arrière]  $\leftarrow$   $\emptyset$ 
5: Lrésultat  $\leftarrow$   $\emptyset$ 

6: labelSource  $\leftarrow$  [source, [0, ..., 0], NULL]
7: source.ajouterLabel(labelSource, avant)
8: T[avant].ajouter(labelSource)

9: labelDest  $\leftarrow$  [destination, [0, ..., 0], NULL]
10: destination.ajouterLabel(labelDest, arrière)
11: T[arrière].ajouter(labelDest)

12: while  $\min(T_{\text{avant}}) + \min(T_{\text{arrière}})$  n'est dominé par aucun  $R \in L_{\text{résultat}}$  do
13:   d  $\leftarrow$  obtenirDirection()
14:   label  $\leftarrow$  T[d].extraireMin()
15:   proprietaire  $\leftarrow$  label.proprietaire()
16:   voisins  $\leftarrow$  proprietaire.voisins()
17:   for all  $n \in \text{voisins}$  do
18:     lien  $\leftarrow$  obtenirLienEntre(proprietaire, n)
19:     nouveauLabel  $\leftarrow$  [n, [label[i] + lien.cout[i]], label]
20:     if  $f(\text{voisin}) \leq \text{distanceMax}$  then
21:       if nouveauLabel n'est dominé par aucun label de n.labels(d) then
22:         n.ajouterLabel(nouveauLabel, d)
23:         T[d].ajouter(nouveauLabel)
24:         if n.label(d') n'est pas vide then

```

```

25:          $resultats \leftarrow \text{combiner}(\text{nouveauLabel}, n.\text{labels}(d'))$ 
26:          $L_{\text{résultat}}.\text{ajouterResultats}(resultats)$ 
27:     end if
28: end if
29: end if
30: end for
31: end while

```

5 Application développée

En nous appuyant sur les données réelles d'OpenStreetMap, nous avons développé une application mettant en œuvre l'algorithme précédemment décrit pour calculer des plus courts chemins pour une carte géographique.

À l'instar des autres applications géographiques, notre application affiche la carte de la ville ainsi qu'un panneau permettant à l'utilisateur de saisir les détails de sa requête.

L'utilisateur renseigne les localisations de départ et d'arrivée, représentées respectivement par un point vert et un point rouge sur la carte. Par ailleurs, il peut définir un seuil qui borne la distance maximale autorisée par rapport à la distance la plus courte. Par défaut, ce seuil est fixé à 10 %.

Il peut également définir ses préférences, notamment les intervalles d'indifférence entre les différentes catégories proposées.

L'utilisateur peut ensuite cliquer sur l'icône « Calculer » pour obtenir l'ensemble des chemins Pareto-optimaux respectant à la fois les contraintes de seuil et ses préférences ordinales.

Les résultats sont affichés sous la forme d'une liste d'itinéraires possibles avec leur distance totale et leurs distances par catégorie (vecteur de coût). En cliquant sur un chemin de cette liste, celui-ci est affiché sur la carte sous forme d'une ligne bleue. Le plus court chemin est quant à lui toujours représenté par une ligne violette. Afin de ne pas surcharger la visualisation, un seul chemin est affiché à la fois, en plus du plus court chemin.

La figure suivante montre l'interface de l'application pour Paris telle qu'elle apparaît à l'utilisateur au démarrage, sans aucune information renseignée. Il s'agit de la version à deux catégories (A, B) de sécurité.

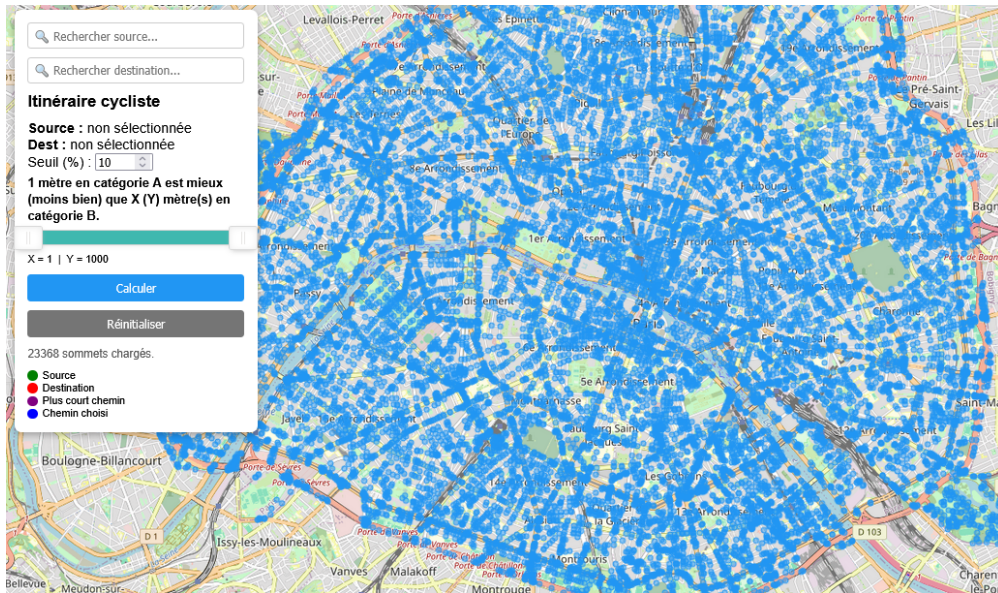


FIGURE 3 – Interface de l'application telle qu'elle apparaît à l'utilisateur au démarrage, sans aucune information renseignée.

La figure suivante illustre un exemple d'itinéraires proposés par l'algorithme après avoir saisi Nation comme point de départ et Bastille comme point d'arrivée.

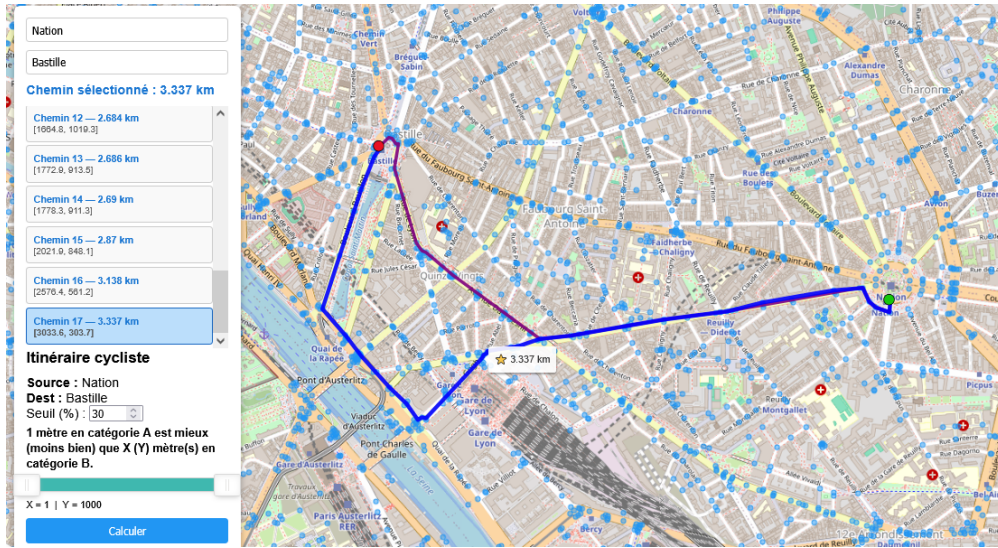


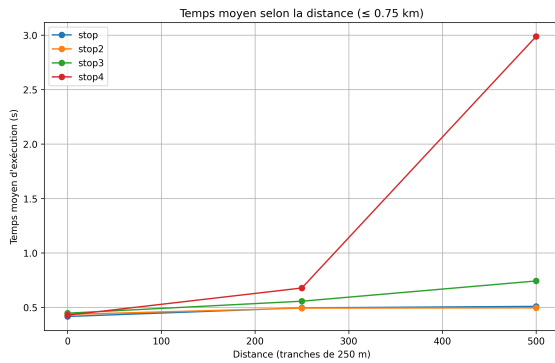
FIGURE 4 – Exemple d’affichage des itinéraires allant de Nation à Bastille renvoyés par l’algorithme.

6 Résultats expérimentaux

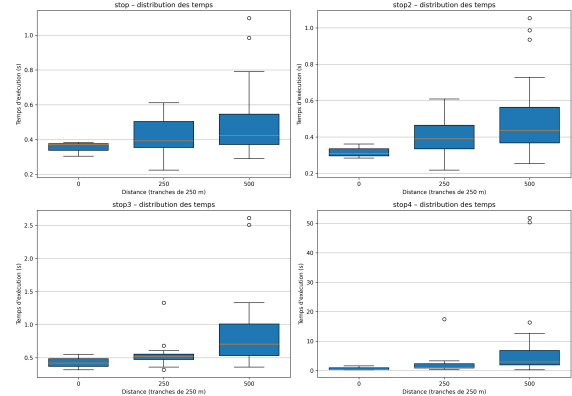
Dans notre implémentation, il y a trois points pour lesquels nous avons différents choix d’approches possibles, et dont nous pouvons comparer les performances (temps d’exécution) : les conditions d’arrêt de l’algorithme principal (mentionnés précédemment), l’heuristique utilisée (*landmarks* + vol d’oiseau ou juste vol d’oiseau) et l’utilisation ou non du polyèdre des jeux de poids. Dans cette section nous présenterons les comparaisons statistiques obtenues concernant ces trois points.

Tous les graphiques sont issus de séries de 100 tests aléatoires avec 3 niveaux de sécurité considérés.

6.1 Condition d’arrêt



(a) Temps moyen : conditions d’arrêt



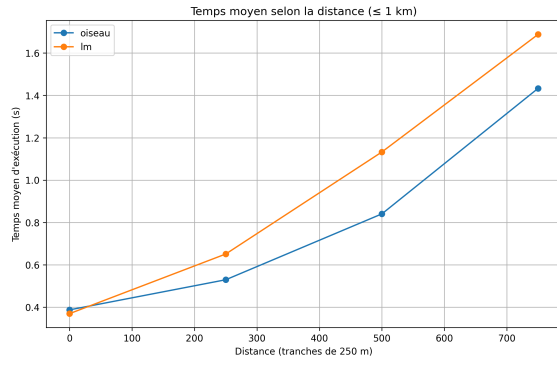
(b) Boxplots : conditions d’arrêt

FIGURE 5 – Comparaison statistique des différentes conditions d’arrêt (moyenne et distribution)

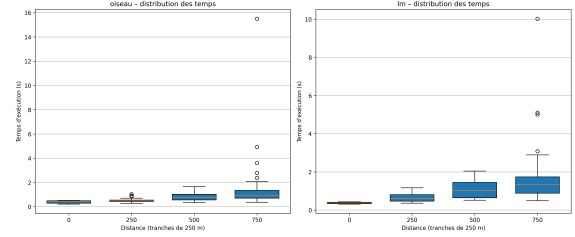
Les graphiques 5 sont assez explicites : la quatrième condition d’arrêt est la plus mauvaise, la troisième est légèrement moins performante que les deux premières, qui sont assez similaires entre elles. Néanmoins, nous nous attendions à ce que la troisième condition d’arrêt soit plus efficace que les deux autres. Cela témoigne potentiellement de soucis d’implémentation.

6.2 Heuristiques

Les graphiques 6 sont une nouvelle fois visuellement clairs : la version avec l’heuristique "vol d’oiseau seul" est plus rapide que celle avec "vol d’oiseau et *landmarks*". Cette conclusion est plutôt inattendue puisque nous pensions que l’ajout de l’heuristique avec les *landmarks* améliorerait les performances. Ce résultat s’explique probablement par le fait que les tests statistiques sont réalisés sur des petits trajets. En effet, le coût de calcul de l’heuristique avec les *landmarks* est compensé quand les distances augmentent (environ à partir de 3km).



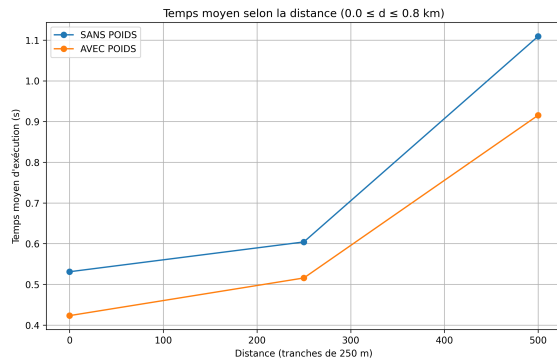
(a) Temps moyen : heuristiques



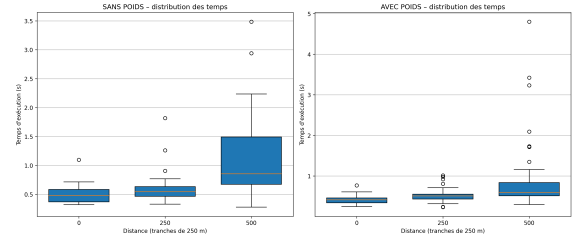
(b) Boxplots : heuristiques

FIGURE 6 – Comparaison statistique des différentes heuristiques

6.3 Jeux de poids



(a) Temps moyen : jeux de poids



(b) Boxplots : jeux de poids

FIGURE 7 – Comparaison statistique des différents jeux de poids

Les graphiques 7 sont assez explicites et cohérents cette fois-ci. La version avec poids est bien plus rapide que sans, mais l'écart n'est pas aussi significatif que souhaité. Cela peut s'expliquer par les choix des coefficients γ et ω qui n'élaguent pas assez de chemins, ou le nombre de classes qui est trop faible. L'écart serait plus important en augmentant le nombre de classes.

Références

- [1] Byeong Seok AHN et Haechurl PARK. “Establishing dominance between strategies with interval judgments of state probabilities”. In : *Omega* 49 (2014), p. 53-59. ISSN : 0305-0483. DOI : [10.1016/j.omega.2014.05.007](https://doi.org/10.1016/j.omega.2014.05.007). URL : <https://doi.org/10.1016/j.omega.2014.05.007>.
- [2] Sérgio BARRETO et al. “Using clustering analysis in a capacitated location-routing problem”. In : *European Journal of Operational Research* 179.3 (2007), p. 968-977. ISSN : 0377-2217. DOI : <https://doi.org/10.1016/j.ejor.2005.06.074>. URL : <https://www.sciencedirect.com/science/article/pii/S0377221706000786>.
- [3] Charles DELORT, Olivier SPANJAARD et Paul WENG. “Committee Selection with a Weight Constraint Based on a Pairwise Dominance Relation”. In : *Algorithmic Decision Theory*. Sous la dir. de Ronen I. BRAFMAN, Fred S. ROBERTS et Alexis TSOUKIÀS. Berlin, Heidelberg : Springer Berlin Heidelberg, 2011, p. 28-41.
- [4] S. DEMEYER, J. GOEDGEBEUR, P. AUDENAERT et al. “Speeding up Martins’ algorithm for multiple objective shortest path problems”. In : *4OR-Q Journal of Operations Research* 11.3 (2013), p. 323-348. DOI : [10.1007/s10288-013-0232-5](https://doi.org/10.1007/s10288-013-0232-5). URL : <https://doi.org/10.1007/s10288-013-0232-5>.
- [5] Matthias EHRGOTT. “Multiobjective Versions of Polynomially Solvable Problems”. In : *Multicriteria Optimization*. Section : Algorithms for shortest path problems. Springer, 2005, p. 221-270. DOI : [10.1007/3-540-27659-9](https://doi.org/10.1007/3-540-27659-9).
- [6] Andrew V. GOLDBERG et Chris HARRELSON. “Computing the shortest path : A search meets graph theory”. In : *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '05)*. USA : Society for Industrial et Applied Mathematics, 2005, p. 156-165.
- [7] Peter E. HART, Nils J. NILSSON et Bertram RAPHAEL. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In : *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), p. 100-107.
- [8] Kathrin KLAMROTH, Michael STIGLMAYR et Julia SUDHOFF SANTOS. *Adapting Polyhedral Dominance Cones to Ordinal Preference Structures*. arXiv preprint. Jan. 2026. arXiv : [2601.02796](https://arxiv.org/abs/2601.02796) [math.OC]. URL : <https://arxiv.org/abs/2601.02796>.
- [9] Roger W. SINNOTT. “Virtues of the Haversine”. In : *Sky and Telescope* 68.2 (1984), p. 159.